
Requirements, Design, and Tasks for raft-rx

FSM-based Raft Implementation

2026-05-12

Contents

Requirements	5
Goal	5
Functional Scope	5
Architecture Requirements	5
R1. Reactive model over rxnet	5
R2. Layer separation	6
R3. C and Python compatibility	6
Transport Requirements	7
R4. Abstract transport	7
R5. Reference transport	7
Persistence Requirements	7
R6. Minimum persistent state	7
R7. Safe writes	7
Application State-Machine Requirements	8
R8. State-machine API	8
R9. Key-value example	8
Observability Requirements	8
R10. Optional and external observability	8
R11. Structured events	9
R12. External shell	9
Non-Functional Requirements	9
R13. Portability	9
R14. Dependencies	9
R15. Determinism and testability	10
R16. Production quality	10
Acceptance Criteria	10
Design	11
Summary	11
Node Model	11

Mapping onto rxnet Phases	13
Latch inputs	13
Evaluate	13
Commit	13
Deferred actions	14
Dump outputs	14
Message Types	14
Log and Commit	14
Cluster reconfiguration	15
Current reconfiguration scope	15
Persistence	16
Transport	16
Clock	17
Observability	17
Proposed Public API	18
Python	18
C	18
Deliberate Differences Between C and Python	18
Tasks	21
Documentation	21
Project Structure	21
Conceptually Shared Raft Core	21
Python	21
C	22
Verification	22

List of Figures

Requirements

Goal

Build a Raft library on top of `./rxnet` in C and Python, aimed at:

- distributed embedded systems with dependency constraints,
- terminal applications on macOS and Linux,
- optional integration with external observability tools,
- deterministic execution based on ticks and state machines.

The result must include a working example of a persistent distributed key-value database.

Functional Scope

The library must implement the core Raft algorithm:

- leader election,
- heartbeats and leadership maintenance,
- log replication,
- majority-based commit,
- ordered application of entries to a state machine,
- persistence of `current_term`, `voted_for`, and the log,
- recovery after restart from persistent storage,
- explicit redirection or rejection of client requests when the node is not the leader.

Architecture Requirements

R1. Reactive model over rxnet

Each Raft node must be modeled as an `rxnet` state machine with at least these roles:

- FOLLOWER
- CANDIDATE
- LEADER

Role transitions must be governed by events latched in the current tick:

- election timeout expiration,
- heartbeat or `AppendEntries` reception,
- vote reception,
- discovery of a higher term.

R2. Layer separation

The solution must clearly separate:

- the Raft protocol core,
- transport,
- persistence,
- application state machine,
- observability.

The core must not depend on a concrete network, shell, or UI implementation.

R3. C and Python compatibility

There must be a C implementation and a Python implementation with aligned semantics for:

- message types,
- term and vote transition rules,
- log validation rules,
- commit semantics,
- transport interface,
- persistence interface,
- application state-machine integration API.

Sharing binaries or an ABI is not required, but the conceptual model and observable behavior must match.

Transport Requirements

R4. Abstract transport

The library must define an abstract transport interface capable of:

- sending Raft messages to a peer,
- receiving all pending messages for a node in one tick,
- injecting client requests,
- avoiding dynamic allocation on critical C runtime paths whenever possible.

R5. Reference transport

At least one deterministic and portable reference transport must be included:

- an in-memory transport for simulation, tests, and local examples.

In addition, the independent-process demos must demonstrate that the core does not depend on the in-memory bus:

- C: pluggable TCP transport with a persistent peer table.
- Python: HTTP transport in the external demo, implemented with the standard library.

Host transports must be addable without modifying the Raft core.

Persistence Requirements

R6. Minimum persistent state

Each node must durably persist:

- `current_term`,
- `voted_for`,
- the Raft log,
- the applied state of the key-value example state machine.

R7. Safe writes

Host persistence must use reasonable atomic or near-atomic writes:

- temporary file plus `rename/replace`,
- readable and auditable format,
- robust recovery after clean restarts.

For embedded targets, the API must allow this persistence layer to be replaced with one adapted to flash, NVRAM, or platform-specific storage.

Application State-Machine Requirements

R8. State-machine API

There must be an application interface with these capabilities:

- validate or accept serialized commands,
- apply committed entries in order,
- query client-visible state,
- load and save persistent state.

R9. Key-value example

A sample state machine must be delivered with these operations:

- `set(key, value)`
- `delete(key)`
- `get(key)`

`get` may be resolved locally against the applied state. `set` and `delete` must enter the leader's log.

Observability Requirements

R10. Optional and external observability

Observability must be fully optional. When it is not enabled, the Raft core must not depend on shell, TUI, or visual libraries.

R11. Structured events

When enabled, each node must be able to emit external structured events for at least:

- role changes,
- term changes,
- votes sent or received,
- RPC reception and transmission,
- entry append and commit,
- command application to the state machine,
- persistence or transport errors.

R12. External shell

A basic, comfortable, extensible external shell must be included for Python. Its minimum responsibilities are:

- display nodes, roles, and terms,
- show log length and `commit_index`,
- show recent events,
- query the key-value state of the example cluster.

The independent-process demos may include a local CLI as an FSM in the same runtime, as long as the core remains separate from that UI.

Non-Functional Requirements

R13. Portability

The C code must compile at least on:

- macOS,
- Linux.

The architecture must remain suitable for embedded-platform ports.

R14. Dependencies

- C: no mandatory external dependencies beyond `rxnet` and the standard C library.

- Python: only the standard library plus local [rxnet](#).
- The external shell must not introduce dependencies into the embedded runtime.

R15. Determinism and testability

The implementation must be executable deterministically in tests:

- controllable clock,
- in-memory transport,
- explicit timeout injection,
- cluster and log state verification.

R16. Production quality

The deliverable must include:

- requirements, design, and task documentation,
- automated tests in C and Python,
- executable examples,
- explicit error handling,
- clear public API.

Acceptance Criteria

The work is considered complete when:

1. [docs/requirements.md](#), [docs/design.md](#), and [docs/tasks.md](#) describe the system coherently.
2. A usable and tested Python library exists.
3. A usable and compilable C library exists.
4. A 3-node in-memory cluster elects a leader and replicates key-value operations.
5. State persists and can be recovered after restart.
6. Observability can be enabled or disabled without affecting the core.
7. Independent C and Python demos start nodes in separate processes and allow membership reconfiguration.

Design

Summary

The solution is organized into five layers:

1. `core`: Raft protocol rules and node modeling as an `rxnet` FSM.
2. `transport`: message and client-request delivery.
3. `storage`: persistence for Raft metadata, log, and applied state.
4. `state_machine`: application logic; in this project, a key-value store.
5. `telemetry`: optional structured event emission for an external shell.

The C and Python implementations share the same baseline semantics, although not the same internal layout. The `joint consensus` reconfiguration described in this document is implemented in the Python and C references.

Node Model

Each node maintains:

- `node_id`
- `config_state`
- `peers`
- `current_term`
- `voted_for`
- `log`
- `commit_index`
- `last_applied`
- `role`
- `leader_id`
- `election_deadline_ms`
- `heartbeat_deadline_ms`

- `next_index[peer]`
- `match_index[peer]`

The `rxnet` FSM models the role and the main Raft events:

- FOLLOWER
- CANDIDATE
- LEADER

The rest of the state lives in the node's user structure and is updated in phase callbacks.

In addition, the Python implementation models two secondary FSMs:

- `CompactionFSM`: `IDLE` → `SNAPSHOT_PENDING` → `COMPACTING` → `IDLE`
- `MembershipFSM`: `STABLE` → `JOINT_PENDING` → `JOINT` → `FINALIZING` → `STABLE`

Membership is no longer a simple list. It is represented as an explicit configuration:

- `old_members`
- `new_members` | `None`
- `configuration_index`

When `new_members` is `None`, the configuration is stable. When `new_members` exists, the node is in `joint consensus`.

The main transition table is:

- FOLLOWER -- `timeout_expired` --> CANDIDATE / `become_candidate`
- FOLLOWER -- `has_append_entries` --> FOLLOWER / `handle_append_entries`
- FOLLOWER -- `has_vote_request` --> FOLLOWER / `handle_vote_request`
- FOLLOWER -- `has_vote` --> FOLLOWER / `ignore_vote`
- CANDIDATE -- `timeout_expired` --> FOLLOWER / `back_to_follower_due_to_timeout`
- CANDIDATE -- `received_majority_votes` --> LEADER / `become_leader`
- CANDIDATE -- `has_append_entries` --> FOLLOWER / `handle_append_entries`
- CANDIDATE -- `has_vote_request` --> CANDIDATE / `handle_vote_request`
- CANDIDATE -- `has_vote` --> CANDIDATE / `handle_vote`
- LEADER -- `has_append_entries` --> FOLLOWER / `handle_append_entries`
- LEADER -- `has_vote_request` --> LEADER / `ignore_vote_request`
- LEADER -- `has_vote` --> LEADER / `ignore_vote`

- `LEADER -- time_for_heartbeat --> LEADER / send_heartbeat`

Event queues are drained in `latch_inputs`, and the FSM guards check whether there are pending events of each type.

Mapping onto rxnet Phases

Latch inputs

In `latch_inputs`, the node:

- reads the current time from an abstract clock,
- drains incoming messages from the transport,
- classifies RPCs and responses into pending queues,
- detects election or heartbeat timeouts,
- accumulates pending client commands,
- computes transition flags for the FSM.

This phase also prepares output buffers and marks pending persistent writes.

Evaluate

`rxnet` evaluates the first enabled transition according to the order in the table above. This enforces clear semantics based on Raft events rather than side-effect callbacks.

Commit

The FSM publishes the new role and executes a deferred action associated with the transition:

- `become_candidate`: increments the term, votes for itself, and sends `RequestVote`,
- `become_leader`: initializes replication indexes and sends an immediate heartbeat,
- `back_to_follower_due_to_timeout`: abandons the current candidacy and resets the timeout,
- `handle_append_entries`: processes the leader RPC,
- `handle_vote_request`: decides whether to grant the vote,
- `handle_vote`: accumulates votes for an active election,
- `send_heartbeat`: sends an empty `AppendEntries`,
- `ignore_vote` and `ignore_vote_request`: consume events that are not relevant to that role.

Deferred actions

Deferred actions are used only for role changes and the initial sends associated with the transition. This keeps the decision about state separate from side effects.

Dump outputs

In `dump_outputs`, the node:

- persists dirty metadata and log data,
- sends pending messages,
- advances leader replication,
- applies committed entries to the state machine,
- emits optional telemetry.

Message Types

Four basic types are defined:

- `REQUEST_VOTE`
- `REQUEST_VOTE_RESPONSE`
- `APPEND_ENTRIES`
- `APPEND_ENTRIES_RESPONSE`

And one internal auxiliary type for clients:

- `CLIENT_COMMAND`

Each message contains:

- `term`
- `source`
- `target`
- RPC-specific fields

`APPEND_ENTRIES` carries zero or more entries. Zero entries is equivalent to a heartbeat.

Log and Commit

Each log entry contains:

- `term`
- `index`
- `leader_id`
- `command`

The leader:

- accepts client commands,
- appends them to its local log,
- replicates them with `AppendEntries`,
- updates `commit_index` when the entry is replicated by a majority in the current term.

Followers:

- validate `prev_log_index` and `prev_log_term`,
- correct conflicts by truncating the incompatible suffix,
- append valid new entries,
- update `commit_index` according to `leader_commit`.

Cluster reconfiguration

Membership changes are not applied with a single entry. They are modeled as two log entries:

- `cluster.enter_joint(C_new)`
- `cluster.leave_joint(C_new)`

The shell exposes high-level operations such as `addnode` and `rmnode`, but internally the leader activates the `MembershipFSM`, which materializes that sequence.

During `joint consensus`, commit calculation requires a double majority:

- majority over `old_members`
- majority over `new_members`

The transition only finishes when the leader considers the members of `C_new` sufficiently caught up.

Current reconfiguration scope

The current implementation covers:

- persistent representation of stable or joint configuration,

- double quorum for commit in joint mode,
- two-log-entry transition,
- operational shell support for `members`, `addnode`, and `rmnode`,
- local provisioning of new nodes in the simulation cluster.

It does not yet cover:

- `InstallSnapshot` for accelerated onboarding of lagging nodes,
- additional leader-eligibility restrictions during reconfiguration.

Persistence

Two abstractions are used:

- `RaftStorage`: persistent protocol state.
- `StateMachineStorage`: persistence for applied application state.

On host systems, the reference persistence is implemented with per-node files:

- `meta.json` or `meta.txt`
- `log.jsonl` or `log.txt`
- `kv.json` or `kv.txt`

`meta.json` also persists:

- `old_members`
- `new_members`
- `configuration_index`
- `compaction_threshold`
- snapshot metadata

Metadata and state-machine writes use a temporary file followed by an atomic replace. In this first version, the log may be rewritten in full to simplify robustness and traceability.

Transport

The transport interface offers:

- `send(message)`
- `recv_for(node_id) -> messages`

- `submit_client_command(node_id, command)`
- `recv_client_commands(node_id)`

The reference implementation is an in-memory bus, deterministic and thread-free, useful for:

- tests,
- simulation,
- a single-process distributed-database example.

In addition to the in-memory transport, the repository includes demo transports for multi-process execution:

- C: `raft_tcp_transport_t`, exposed in `raft/raft_tcp_transport.h`, with a TCP listener, persistent peer table, and `join/merge` protocol.
- Python: `HttpTransport` inside `python/examples/demo_app`, with REST endpoints for Raft messages, client commands, and membership changes.

The API leaves room for future transports over UDP, Unix sockets, or RTOS queues.

Clock

Time is not read directly from the system inside the core. It is abstracted through a clock:

- Python: `Clock.now_ms()`
- C: callback or integrator-updated field

This enables deterministic tests and embedded deployments with custom timers.

Observability

In Python, observability is implemented with an optional `TelemetrySink`. If it is `NoOp`, there is no external dependency cost.

Events are emitted as lightweight structures and may be serialized as JSON Lines for consumption by an external shell.

The generic Python shell:

- is an independent process,
- reads events from JSONL files,
- presents an aggregated cluster view,

- is not part of the node runtime.

In C, the available observability is optional `rxnet` tracing (`RX_TRACE_ENABLE`) and traced examples that export `trace.bin`.

Proposed Public API

Python

- `raft_rx.RaftApplication`
- `raft_rx.Command`, `LogEntry`, `Message`, `MessageKind`
- `raft_rx.ManualClock`, `SystemClock`
- `raft_rx.MemoryTransport`
- `raft_rx.JsonFileStorage`
- `raft_rx.JsonlTelemetrySink`, `NullTelemetrySink`
- `raft_rx.NodeConfig`, `RaftNode`, `Role`
- `raft_rx.RaftCluster`
- `raft_rx.RaftShell`

C

- `include/raft/raft.h`
- `include/raft/raft_kv_app.h`
- `include/raft/raft_tcp_transport.h`

`libraft_rx.a` contains the core, in-memory transport, file persistence, TCP transport, and sample KV state machine.

Deliberate Differences Between C and Python

Python prioritizes:

- ergonomics,
- simulation,
- external shell,
- easy state inspection.

C prioritizes:

- explicit API,
- portability to constrained environments,
- reasonable fixed buffers and capacities,
- no additional dependencies.

Protocol semantics must remain aligned, even when the internal representation is not identical.

Tasks

Documentation

- ☒ Write system requirements.
- ☒ Write the layered design and mapping onto [rxnet](#).
- ☒ Define the scope of the first production-ready version.

Project Structure

- ☒ Create the Python code tree.
- ☒ Create the C code tree.
- ☒ Add the main README and build/test commands.

Conceptually Shared Raft Core

- ☒ Define message types and log entries.
- ☒ Define term, vote, and commit semantics.
- ☒ Implement log validation rules.
- ☒ Implement recovery from persistent storage.

Python

- ☒ Create the [raft_rx](#) package.
- ☒ Implement a deterministic clock for tests.
- ☒ Implement the in-memory transport.
- ☒ Implement file-based persistence.
- ☒ Implement no-op and JSONL [TelemetrySink](#).
- ☒ Implement the persistent key-value state machine.

- ☒ Implement the Raft node over `rxnet.fsm`.
- ☒ Implement a simulation cluster.
- ☒ Create an executable distributed-database example.
- ☒ Create an external shell for cluster operation and inspection.
- ☒ Add tests for election, replication, and recovery.
- ☒ Implement stable/joint cluster configuration and the secondary membership FSM.
- ☒ Expose reconfiguration in the shell with `members`, `addnode`, and `rmnode`.
- ☒ Add an independent demo with HTTP transport, local CLI, `join`, and `merge`.

C

- ☒ Create the `raft` library.
- ☒ Implement public types and limits.
- ☒ Implement the in-memory transport.
- ☒ Implement reference file-based persistence.
- ☒ Add optional tracing through `rxnet (RX_TRACE_ENABLE)` and an exportable example.
- ☒ Implement the persistent key-value state machine.
- ☒ Implement the Raft node over `rxnet/fsm.h`.
- ☒ Create a 3-node cluster example.
- ☒ Add tests for election, replication, and restart.
- ☒ Port `joint consensus` reconfiguration to the C library.
- ☒ Add an independent demo with TCP transport, local CLI, `join`, and `merge`.

Verification

- ☒ Run Python tests.
- ☒ Compile C.
- ☒ Run C tests.
- ☒ Verify the key-value example in both languages.
- ☒ Update the documentation with the final state of the deliverable.